

Design de Alto Padrão e Segurança: O Checklist Que Falta no Seu Produto Digital

Introdução: por que seu produto “parece” barato ou “está” vulnerável sem você perceber

Existem dois tipos de defeito que matam um produto digital antes mesmo de ele ter uma chance real no mercado. O primeiro é visível e mata devagar: o visitante entra no site, no app ou no painel, passa os olhos por três segundos e sente — sem conseguir explicar exatamente por quê — que aquilo “parece amador”. Ele não fecha a aba porque encontrou um erro. Fecha porque uma parte muito antiga do cérebro humano, treinada a vida inteira a associar capricho visual com confiabilidade, deu um veredito instantâneo: isso aqui não é sério.

O segundo defeito é invisível e mata de uma vez só: uma falha de segurança que ninguém viu, porque ninguém testou de propósito. Ela não aparece em nenhum teste manual comum, porque testes manuais seguem o caminho feliz — a pessoa testando é o próprio time, tentando fazer o produto funcionar, não tentando quebrá-lo. Um invasor, ao contrário, não segue o caminho feliz. Ele troca um ID na URL, manda um campo extra no corpo da requisição, chama a API direto sem passar pela tela bonita. E se a trava que devia estar ali só existe no visual — um botão escondido, uma condição verificada só no navegador — ela simplesmente não existe.

Os dois problemas têm uma coisa em comum: o dono do produto quase nunca os vê sozinho. Design ruim não aparece como um erro no console; aparece como uma taxa de conversão baixa que ninguém sabe explicar. Segurança ruim não aparece como um alerta piscando; aparece como silêncio, até o dia em que aparece como um incidente. Os dois são invisíveis para quem construiu o produto porque quem construiu já viu aquela tela cem vezes e já sabe, de cabeça, o que cada botão faz — e é exatamente essa familiaridade que impede de ver com os olhos de um visitante novo ou de um atacante mal-intencionado.

Isto custa caro de duas formas diferentes. O design ruim custa receita todos os dias, silenciosamente: cada visitante que decide “não parece confiável o suficiente” e sai sem comprar é uma venda que nunca vai aparecer em nenhum relatório de erro — porque não é um erro, é uma escolha invisível de alguém que nunca soube que fez essa escolha. A segurança ruim custa de forma diferente: nada acontece por meses, às vezes anos, e quando acontece, acontece tudo de uma vez — dado de um cliente vazado para outro, conta invadida, banco de dados exposto, ou pior, uma trava de cobrança furada silenciosamente por semanas antes de alguém notar a discrepância no caixa.

Este ebook existe para tirar os dois problemas do território do “eu acho que está bom” e colocá-los no território do checklist verificável. A Parte 1 cobre o que separa visualmente um produto que parece ter custado R200*de um que parece ter custado* R10 mil — e por que essa diferença

não é sobre gastar mais em design, é sobre tomar um conjunto específico de decisões que qualquer pessoa, com qualquer orçamento, pode tomar. A Parte 2 cobre o checklist técnico de segurança que qualquer produto que lida com login, dado de usuário ou pagamento precisa passar antes de ir ao ar — explicado para quem não é especialista em segurança, mas sem perder a substância técnica real, porque um checklist de segurança “simplificado demais” é um checklist que não protege nada.

No final, as duas partes se juntam num único checklist de “antes de lançar” — porque na prática é assim que funciona: o produto só está pronto quando as duas frentes estão cobertas ao mesmo tempo.

PARTE 1 — DESIGN DE ALTO PADRÃO

Capítulo 1 — O que realmente separa um site de R 200deumdeR10 mil

A primeira coisa a entender é que a diferença não é o preço do template, nem a ferramenta usada para construir, nem quantas horas foram gastas. Dois produtos podem ser feitos com a mesma stack, no mesmo tempo, e um vai parecer premium e o outro vai parecer genérico — porque a diferença está numa lista específica e verificável de decisões. Cada item abaixo é uma decisão binária: ou foi tomada de propósito, ou não foi tomada e o produto herdou o padrão genérico por omissão.

1. Ponto de vista de design próprio. A pergunta que separa um produto premium de um template com a logo trocada é simples: se um concorrente copiasse e colasse sua página inteira e só trocasse o texto, ela ainda faria sentido pra ele? Se a resposta for sim, seu design não tem ponto de vista — é neutro o suficiente para servir qualquer negócio, o que na prática significa que não comunica nada sobre o seu negócio específico. Um design com ponto de vista carrega decisões que só fazem sentido para aquele produto: uma paleta que reflete o tom da marca, uma forma de organizar informação que reflete como aquele produto específico resolve o problema, uma escolha visual que seria estranha em qualquer outro contexto.

2. Tipografia com hierarquia forte. A maioria dos sites amadores usa uma fonte só, em três tamanhos parecidos — título um pouco maior, subtítulo um pouco menor, corpo do mesmo jeito. O olho não consegue escanear a página porque tudo tem o mesmo peso visual. Um design com hierarquia de verdade faz o título, o subtítulo e o corpo serem claramente diferentes — não só em tamanho, mas em peso (bold vs. regular), às vezes em família tipográfica, sempre em função: o título existe para ser lido em meio segundo, o corpo existe para ser lido com atenção. Se as duas coisas competem visualmente pela mesma atenção, nenhuma vence.

3. Paleta restrita. Este é talvez o erro mais fácil de cometer e mais fácil de corrigir: usar cores demais. O padrão de alto padrão é entre três e cinco cores no total — incluindo neutros (branco, preto, cinzas). Cada cor extra além disso não soma, briga: o olho não sabe para onde olhar porque tudo grita ao mesmo tempo. Um produto premium costuma ter uma cor de fundo, uma de texto, uma primária e no máximo uma ou duas de destaque usadas com moderação — e é exatamente essa restrição que faz a cor de destaque funcionar, porque ela só aparece quando importa.

4. Hierarquia que respira. Espaço em branco generoso não é “espaço desperdiçado” — é a ferramenta mais barata e mais subestimada de comunicar sofisticação. Densidade visual — botões colados, textos empilhados, seções sem respiro — comunica pressa, não eficiência. Produtos caros de verdade (o design de uma marca de luxo, de um banco premium, de um software enterprise bem pago) quase sempre usam mais espaço em branco do que pareceria “necessário” à primeira vista. Isso é proposital: o espaço em branco diz “temos tempo e recurso sobrando para não precisar espremer tudo”.

5. Imagens com intenção. Nada denuncia um produto amador mais rápido que uma foto de banco de imagens genérica — aquela pessoa sorrindo forçado num escritório que não existe, apertando a mão de alguém num terno que não bate com o resto do site. Se você não tem uma imagem real (foto do produto de verdade, screenshot real da interface, foto do time real), a regra é: melhor não ter imagem nenhuma do que ter uma falsa. Um espaço vazio bem desenhado, ou uma ilustração/ícone simples e consistente com o resto da identidade, comunica mais confiança que uma foto de banco que qualquer visitante já viu em outros dez sites.

6. Movimento com propósito. Animação e microinteração existem para ajudar a pessoa a entender alguma coisa — revelar um conteúdo no momento certo, guiar o olho para onde a próxima ação deve acontecer, dar feedback de que um clique foi registrado. O erro comum é usar movimento como decoração, “porque parece moderno”: elementos que sobem flutuando sem razão, transições longas demais que atrasam a leitura, hover effects que não comunicam nada além de “isto é clicável” (coisa que já devia estar clara pelo próprio design). A pergunta certa antes de adicionar qualquer animação é: isso ajuda a pessoa a entender algo mais rápido, ou só está ali porque fica bonito no vídeo de demonstração?

7. Mobile desenhado, não encolhido. A maioria do tráfego de qualquer produto hoje é mobile, e a maioria dos produtos amadores trata o mobile como uma versão “espremida” do desktop — os mesmos elementos, na mesma ordem, só menores. Um produto de alto padrão desenha o layout mobile como uma versão própria: a ordem das seções pode mudar, elementos que fazem sentido lado a lado no desktop podem precisar virar outra coisa no mobile (não só empilhar), e a área de toque de cada botão precisa ser grande o suficiente para um polegar real, não um cursor de mouse.

8. O caro invisível. Existe uma categoria inteira de qualidade que ninguém “vê” conscientemente, mas todo mundo sente quando falta: performance real (a página carrega rápido, sem travar no scroll), contraste de texto acessível (no mínimo o padrão AA, que significa

uma relação de contraste de pelo menos 4.5:1 entre texto e fundo), HTML semântico (usar as tags certas para cada tipo de conteúdo, o que também ajuda em acessibilidade e em SEO), e meta tags/Open Graph configuradas corretamente (o que faz o link do seu produto aparecer bonito quando alguém cola no WhatsApp ou no Twitter, em vez de aparecer sem imagem e sem descrição). Nenhum desses itens aparece como um elemento visual que alguém aponta e elogia — mas a ausência de qualquer um deles é sentida como “algo está errado aqui” mesmo por quem não sabe nomear o quê.

Capítulo 2 — Os princípios de um produto que as pessoas compartilham

O capítulo anterior foi sobre estética. Este é sobre estrutura de negócio e de conversão — as decisões que fazem alguém não só achar seu produto bonito, mas decidir comprar, e depois decidir contar para outra pessoa. Nem todo princípio abaixo se aplica a todo produto: alguns são escolhas de modelo de negócio (por exemplo, “evite plano grátis para sempre” não vale para todo tipo de produto), não regras absolutas. Trate esta lista como uma bússola — um conjunto de perguntas a fazer antes de decidir o oposto — não como uma lei. Onde um princípio conflitar com uma decisão de produto já bem fundamentada, a decisão de produto vence; mas vale sempre perguntar “por que estamos indo contra isso, e essa razão é boa o suficiente?”

Estrutura e oferta

Um plano gratuito para sempre atrai uma quantidade desproporcional de gente que nunca vai pagar — e cada uma dessas contas custa suporte e infraestrutura de graça, indefinidamente. Quando o modelo de negócio permitir, uma avaliação com prazo definido (trial) ou um paywall direto tende a filtrar melhor do que “grátis para sempre”. Na hora de estruturar o preço, três opções — a barata, a recomendada e a completa, no formato “Bom, Melhor, Ótimo” — convertem melhor do que cinco ou seis opções: mais alternativas geram mais fricção de decisão, e diante de fricção demais, muita gente simplesmente não decide nada. O preço também precisa estar visível — um link “Preço” no menu principal, não escondido atrás de um formulário de “fale conosco” sem necessidade real, a não ser que o modelo de vendas genuinamente exija uma conversa antes do preço (vendas enterprise, por exemplo). Um detalhe contraintuitivo: cobrar mais que o concorrente direto costuma converter melhor que cobrar menos, porque preço alto sinaliza qualidade e ninguém compartilha “achei a segunda opção mais barata do mercado” como se fosse um elogio. E, onde o modelo permitir, pagamento único costuma ser mais fácil de vender do que assinatura — mas só vale abandonar a recorrência se o negócio realmente não depender dela para sobreviver no longo prazo.

Visual e atenção

Na prática de conversão, a regra das cores fica ainda mais restrita do que no capítulo de design puro: idealmente três cores no total — uma de base (fundo), uma de texto, e uma única cor de destaque reservada exclusivamente para a ação principal (o botão de comprar, de cadastrar, de agendar). Cada seção da página deve comunicar uma ideia e parar — misturar duas mensagens na mesma seção dilui as duas. A imagem que aparece quando alguém compartilha o link do seu produto (a imagem Open Graph) é, na prática, vista mais vezes do que a própria página — porque a maioria das pessoas vê o link compartilhado antes de decidir clicar. Trate essa imagem como a thumbnail de um vídeo: ela precisa, sozinha, comunicar o suficiente para gerar o clique. E o rodapé da página não deveria ser um encerramento burocrático de “todos os direitos reservados” — deveria valer a pena ler ou compartilhar, porque a maioria de quem visita uma página não compra na primeira visita, mas pode compartilhar; a última coisa que a pessoa vê antes de sair é o que ela lembra.

Copy e mensagem

Números vencem adjetivos, sempre: “economiza 4 horas por semana” convence muito mais do que “rápido e eficiente”, porque um número é verificável e um adjetivo é uma alegação vazia que qualquer concorrente também usa. O título principal precisa ser entendido por alguém de fora do nicho, numa única leitura — complexidade na primeira frase mata a curiosidade antes dela nascer. Elimine palavras fracas do vocabulário da página: “a maioria”, “muitos”, “geralmente”, “raramente” enfraquecem qualquer alegação porque ninguém sabe o que elas significam de verdade; prefira sempre uma afirmação concreta e verificável no lugar. Assim como no design, a copy também precisa ter ponto de vista: se um concorrente pudesse colar seu texto de vendas no site dele sem soar estranho, o texto é genérico demais. Uma técnica poderosa e barata é roubar a melhor copy dos próprios clientes — eles descrevem o valor do produto com palavras melhores do que o time interno normalmente consegue, porque descrevem a partir da própria dor, não do jargão do produto. O nome do produto em si deveria ser fácil de lembrar e de pronunciar — palavras que as pessoas já conhecem, sem trocadilho que precise de explicação para ser entendido.

Duas técnicas de copy separam quem escreve bem de quem escreve muito bem: primeiro, o título de maior impacto costuma emocionar, não listar recursos — as pessoas lembram de como se sentiram lendo uma página, não da lista de funcionalidades; segundo, descrever o problema da pessoa melhor do que ela mesma consegue descrever, antes de sequer mencionar a solução, é a forma mais eficaz de gerar confiança — é empatia demonstrada antes da venda, e demonstra que quem escreveu realmente entende a dor de quem está lendo. Na mesma linha, venda sempre o desejo humano por trás da funcionalidade, não a funcionalidade em si: mais tempo livre, mais dinheiro, mais status, menos dor — a funcionalidade é só o veículo que entrega isso, e é o desejo que motiva a decisão de compra, não a lista técnica. Por fim, quando fizer sentido, compare diretamente com o concorrente: a decisão real de quem está lendo

raramente é “ter ou não ter” o produto, é “trocar ou não trocar” o que já usa — uma tabela simples e honesta com o que realmente importa para quem decide facilita muito essa escolha.

Estrutura da página

Mostrar o produto antes de explicá-lo comunica mais em cinco segundos do que três parágrafos de texto explicativo — uma demonstração visual, um vídeo curto, um GIF do fluxo real, tudo isso vale mais que adjetivos. Um produto que faz uma coisa só, e é conhecido exatamente por essa coisa, tem mais chance de ser lembrado e recomendado do que um “canivete suíço” que faz de tudo — ninguém indica o canivete suíço para um amigo, indica a ferramenta que resolveu exatamente aquele problema específico. A seção principal da página (o hero, a primeira tela que a pessoa vê) precisa vender sozinha, porque a maioria dos visitantes nunca rola a página além dela — se a proposta de valor não está clara ali, ela nunca vai ser lida em lugar nenhum.

Dentro de cada tela, deveria existir uma única chamada para ação — cada botão extra além do principal é mais uma decisão que a pessoa precisa tomar, e diante de opções demais, o comportamento comum não é escolher a “melhor”, é não escolher nenhuma. O texto desse botão deveria dizer exatamente o que acontece a seguir: “Começar” não diz nada de concreto; “Analisar meu site agora” diz exatamente o próximo passo e reduz a ansiedade de clicar em algo desconhecido. Sempre que o modelo permitir, deixe a pessoa experimentar antes de pagar — esconder o melhor recurso atrás de um paywall logo na primeira visita é pedir compromisso antes de entregar valor, o que inverte a ordem que gera confiança. E nunca lance sem pelo menos uma prova social real — um depoimento, um número concreto de resultado, um caso de uso documentado — porque pedir a confiança de um estranho sem nenhuma evidência de que outras pessoas já confiaram é pedir demais logo de cara. Por fim, um bom teste de prontidão: você consegue descrever o produto em menos de dez palavras? Se você não consegue, o usuário também não vai conseguir explicar para outra pessoa — e é assim, boca a boca, que a maioria dos produtos realmente cresce.

Direção e originalidade

Construir em cima de uma onda que já existe — uma tendência, uma tecnologia nova, um problema do qual as pessoas já estão falando ativamente — funciona como marketing embutido, porque a atenção já está ali antes mesmo do produto existir. Mostrar o rosto e a voz de quem criou o produto — um vídeo curto e pessoal do fundador falando diretamente com a câmera — costuma converter mais do que um vídeo institucional genérico ou uma lista fria de recursos, porque as pessoas confiam em pessoas antes de confiar em marcas. E, no fim, o critério mais difícil e mais valioso: fazer algo que as pessoas realmente nunca viram antes. Ninguém compartilha mais um clone do que já existe — compartilhamento genuíno nasce de surpresa, e surpresa exige originalidade real, não uma versão levemente diferente de algo que o mercado já viu cem vezes.

Capítulo 3 — Design system: o mínimo que todo produto precisa definir

Antes de escrever a primeira linha de estilo visual — seja em código, seja numa ferramenta visual — vale ter um punhado de decisões tomadas e documentadas, mesmo que informalmente num documento simples. Isso evita o problema mais comum de projetos sem essa disciplina: cada tela nova reinventa levemente as decisões da anterior, e depois de algumas semanas o produto tem sete tons de azul diferentes e nenhum deles foi escolhido de propósito.

Cor. Defina de quatro a seis valores nomeados — não “azul”, mas o código hexadecimal exato — incluindo pelo menos uma cor de fundo, uma de texto, uma primária e uma de destaque/ação. Uma vez definidos, esses valores deveriam ser os únicos usados em qualquer lugar do produto; qualquer cor nova que “parece precisar” deveria primeiro passar pela pergunta “por que nenhuma das cores já definidas serve aqui?”.

Tipografia. No máximo duas famílias tipográficas — uma de destaque para títulos, usada com moderação, e uma de corpo, usada para tudo que precisa ser lido com atenção. Defina uma escala tipográfica consistente entre os níveis de texto (uma razão comum é 1.25 ou 1.333 entre um nível e o próximo — ou seja, cada tamanho de fonte é aproximadamente esse múltiplo do anterior), e nunca deixe o corpo de texto principal menor que 16 pixels, porque abaixo disso a leitura em telas pequenas fica desconfortável e a acessibilidade cai.

Espaçamento. Use uma escala fixa de espaçamento — múltiplos de 4 pixels ou de 8 pixels são os mais comuns — em vez de valores arbitrários espalhados pelo código ou pela ferramenta de design (13px aqui, 22px ali). Uma escala fixa faz o produto parecer visualmente consistente sem que ninguém precise pensar conscientemente sobre isso a cada elemento novo.

Contraste. Todo par de texto e fundo usado no produto precisa atingir no mínimo o padrão de acessibilidade AA — uma relação de contraste de pelo menos 4.5:1 para texto normal (e um pouco menos para texto grande). Isso não é só uma questão de inclusão para pessoas com baixa visão — texto de baixo contraste também é mais difícil de ler para qualquer pessoa em ambientes com luz forte, como usar o celular ao ar livre.

Grid. Defina um grid de referência — 12 colunas é o padrão mais comum, ou o que a ferramenta/framework usar por padrão — com breakpoints (os pontos onde o layout muda de comportamento entre mobile, tablet e desktop) definidos com antecedência e, principalmente, testados de verdade em dispositivos reais ou em um simulador de dispositivo real, não apenas “encolhendo” a janela do navegador no computador.

Componentes core. Botão (com todos os estados visuais definidos: normal, ao passar o mouse, ao clicar, desabilitado), campo de formulário, card e modal/janela de diálogo — esses quatro componentes aparecem em praticamente qualquer produto digital e deveriam ser

definidos uma única vez e reutilizados sempre, em vez de recriados ligeiramente diferentes em cada tela nova.

Capítulo 4 — Antes de construir: questione o plano

Para decisões de design ou de produto que são grandes ou ambíguas — um reposicionamento de marca, uma página de vendas nova, uma mudança de estrutura de preço, um redesign completo — vale a pena parar antes de começar a executar e interrogar o próprio plano. Na prática, isso significa percorrer cada ramo relevante da decisão (que público estamos priorizando nesta página? que objeção estamos resolvendo primeiro? que ação única queremos que aconteça aqui?), resolver as dependências entre essas escolhas uma de cada vez, e só depois seguir para a execução visual.

Existe uma pergunta específica que vale repetir em cada decisão de design tomada: “isso é uma escolha deliberada para este produto específico, ou é o padrão genérico que qualquer pessoa geraria automaticamente para um briefing parecido com o meu?” Sempre que a resposta honesta for “o padrão genérico”, vale voltar e escolher de novo — porque o padrão genérico é, por definição, o que qualquer concorrente também teria escolhido, e é exatamente esse tipo de escolha que faz um produto parecer intercambiável com qualquer outro do mercado.

Capítulo 5 — Checklist final de design antes de publicar

Antes de considerar o visual e a copy “prontos”, passe pela lista abaixo item por item. Cada linha é uma pergunta de sim ou não — se a resposta for não, o item não está pronto, independente de quão bem o resto da página ficou.

- ☐ Um concorrente não conseguiria copiar e colar esta página inteira e ela ainda fazer sentido para o negócio dele.
- ☐ A paleta usa no máximo de três a cinco cores, com uma claramente reservada para a ação principal.
- ☐ O título principal é entendível por alguém de fora do nicho, numa única leitura, sem precisar de contexto adicional.
- ☐ Existe pelo menos uma prova social real — depoimento, número verificável, caso documentado — visível antes do lançamento.
- ☐ Cada tela tem uma única chamada para ação principal, com um texto de botão que descreve exatamente o que acontece a seguir.
- ☐ A imagem de compartilhamento (Open Graph) foi desenhada como uma thumbnail pensada para gerar clique, não é um print de tela aleatório.

- ☐ O layout foi testado de verdade em um dispositivo móvel real, não só redimensionando a janela do navegador no computador.
 - ☐ O contraste entre texto e fundo atinge pelo menos o padrão AA em todas as combinações de cor usadas na página.
 - ☐ O preço, quando existir, está visível e fácil de encontrar, com no máximo três opções de plano.
 - ☐ O rodapé ou o fechamento da página vale a pena ler ou compartilhar, não é só um bloco burocrático de direitos autorais.
-

PARTE 2 — SEGURANÇA

Capítulo 6 — Como pensar em segurança sem ser um especialista

Antes de entrar no checklist técnico, vale alinhar uma forma de pensar sobre segurança que funciona mesmo para quem nunca trabalhou com isso profissionalmente. A ideia central é simples: qualquer regra que existe apenas na interface visual — um botão escondido, uma opção que só aparece para quem está logado, uma validação que só roda no navegador — não é uma trava de segurança. É apenas uma conveniência de interface. Um visitante com conhecimento mínimo de ferramentas de desenvolvedor (as mesmas que vêm embutidas em qualquer navegador moderno, acessíveis por qualquer pessoa) pode chamar exatamente a mesma requisição que o botão chamaria, sem passar pelo botão, sem passar pela validação visual, sem passar por nada que só existe do lado de quem vê a tela. A regra de ouro de toda esta parte do livro é: toda trava que realmente importa tem que existir no servidor, não só na tela.

A segunda ideia central é sobre prioridade. Segurança não é uma lista de itens igualmente importantes — alguns erros custam uma reclamação de usuário, outros custam o banco de dados inteiro exposto. Este checklist está organizado, de propósito, do que é mais caro de errar para o que é mais barato de errar, para que quem tem tempo limitado saiba exatamente por onde começar: controle de acesso primeiro, sempre.

A terceira ideia, e talvez a mais relevante para quem constrói produtos hoje com ajuda intensa de ferramentas de geração automática de código: nenhum código está seguro só porque “funcionou” no teste manual. Um teste manual comum segue o caminho feliz — a pessoa testando está tentando fazer o sistema funcionar, não tentando quebrá-lo. Um atacante nunca segue o caminho feliz. Ele troca um número na URL, manda um campo a mais no formulário,

chama a rota interna diretamente. Passar no teste normal não é evidência de estar seguro contra alguém que está tentando ativamente furar o sistema — são dois testes completamente diferentes.

Capítulo 7 — Controle de acesso e autorização (o mais caro de errar)

Esta é, disparado, a categoria de erro mais cara e mais comum em produtos digitais — e a boa notícia é que, uma vez entendida, ela também é uma das mais fáceis de verificar sistematicamente.

Toda tabela que guarda dado de usuário precisa ter uma política de restrição por linha ativa e correta. A maioria dos bancos de dados modernos oferece um mecanismo chamado Row Level Security (segurança em nível de linha) ou equivalente — uma regra, configurada no próprio banco, que decide quem pode ler ou escrever cada linha de uma tabela, independente do que o código da aplicação faz. O erro comum não é esquecer de ativar esse mecanismo — muitos frameworks modernos até avisam ou até ativam por padrão. O erro comum é ativar e nunca conferir se a regra escrita realmente restringe alguma coisa. Não presuma que está certo porque “o framework é seguro por padrão” — leia a regra de verdade.

Nenhuma política pode usar uma condição sempre-verdadeira. Uma regra escrita como “permitir se verdadeiro” (`using (true)`, `if True`, ou um middleware que só checa “a pessoa está logada” sem checar “esta pessoa é dona deste dado específico”) na prática não restringe nada — ela expõe a tabela inteira para qualquer usuário autenticado, e às vezes até para visitantes anônimos. Esse é, de longe, o erro de segurança mais comum e mais caro em bancos de dados configurados às pressas: a regra existe, mas não protege.

A restrição de edição precisa cobrir colunas, não só linhas. Este ponto é sutil e é onde a maioria das pessoas erra mesmo depois de configurar a regra de linha corretamente. Uma regra do tipo “o dono só pode editar a própria linha” impede que outra pessoa edite o dado — mas não impede que o próprio dono edite qualquer coluna daquela linha, incluindo colunas sensíveis como plano de assinatura, saldo, nível de permissão ou papel de acesso. Se existe uma coluna que o cliente não deveria conseguir editar por conta própria — como o próprio plano de pagamento ou o próprio nível de administrador — essa coluna precisa de uma proteção separada: uma regra específica que bloqueie a edição daquela coluna, um mecanismo que reverte qualquer alteração indevida automaticamente, ou simplesmente restringir a edição dela para acontecer só por trás do servidor, nunca diretamente pelo cliente.

Referência direta insegura a objetos (o clássico “troca o número na URL”). Todo endereço que recebe um identificador — o número de um pedido, o ID de um usuário — precisa verificar que aquele recurso pertence a quem está pedindo, não apenas que o identificador existe no banco. Trocar o número na barra de endereço não pode, em nenhuma circunstância, revelar o

dado de outra pessoa. Este é um dos erros mais fáceis de testar sozinho: pegue duas contas de teste, pegue o número do pedido/documento/recurso de uma delas, e tente acessar logado na outra.

Atribuição em massa. Quando uma rota de criação ou edição aceita o objeto inteiro que vem do formulário ou da requisição, sem filtrar quais campos são permitidos, ela abre uma porta perigosa: nada impede que alguém, chamando a API diretamente (não pelo formulário visual), envie campos extras que jamais deveriam poder ser definidos por um cliente — como marcar a própria conta como administradora, ou zerar o próprio preço de um item. A defesa é sempre a mesma: fazer uma lista explícita dos campos que o cliente tem permissão de enviar, e nunca simplesmente repassar o corpo inteiro da requisição direto para o banco.

Esconder um recurso pago só na tela não é controle de acesso. Se um recurso é “escondido” apenas ocultando o botão ou o menu na interface, qualquer pessoa capaz de chamar a mesma API diretamente — o que não exige nenhuma habilidade avançada — consegue usar aquele recurso de graça. A trava real precisa estar no servidor: na regra do banco de dados, no middleware da rota, ou dentro da própria lógica que processa a requisição.

Sistemas com múltiplas contas/empresas/organizações. Qualquer consulta que lista ou altera dado pertencente a uma “conta”, “empresa” ou “organização” precisa filtrar sempre pelo grupo do usuário autenticado — nunca confiar num identificador de grupo enviado pelo próprio cliente sem cruzar com a sessão real de quem está logado.

Rotas administrativas e internas precisam de autorização real, não só autenticação. Um endereço como um painel administrativo ou uma rota de uso interno precisa checar não só “existe uma sessão válida”, mas também “esta sessão pertence a alguém com o papel certo para estar aqui”.

Tokens, convites e chaves de acesso compartilhável precisam ser gerados de forma imprevisível. Use sempre um gerador criptográfico de números aleatórios — os disponíveis em qualquer linguagem de programação moderna — nunca uma função aleatória comum, um timestamp, ou um número sequencial previsível. Um código de convite previsível pode ser adivinhado ou percorrido sistematicamente por alguém tentando várias combinações.

Capítulo 8 — Autenticação e sessão

Autenticação é a porta de entrada, e alguns erros aqui são particularmente fáceis de cometer sem perceber.

O token que prova que alguém está logado nunca deveria ficar num tipo de armazenamento do navegador que é livremente legível por qualquer script rodando na página. Um cookie configurado como “somente HTTP”, “seguro” e com a política de mesmo site (`httpOnly` , `Secure` , `SameSite`) é a forma correta, porque esse tipo de armazenamento não pode ser lido por JavaScript — mesmo que um script malicioso consiga rodar na página (por um ataque

de injeção de código, uma extensão de navegador comprometida, ou até uma biblioteca de terceiros que foi silenciosamente comprometida), ele não consegue roubar o token de sessão guardado dessa forma. Guardar o token num armazenamento comum e livremente legível pelo JavaScript da página é abrir essa porta desnecessariamente.

Senhas nunca devem existir em texto puro em lugar nenhum — nem no banco de dados, nem em um log, nem num comentário de código, nem numa massa de dados de teste, nem no histórico do controle de versão. A forma correta é sempre um algoritmo de hash desenhado especificamente para senhas (os dois mais usados hoje são bcrypt e argon2), nunca um hash reversível ou uma criptografia simétrica comum — a diferença é que um hash de senha correto é matematicamente impossível de reverter de volta para a senha original, mesmo por quem projetou o sistema.

Links de redefinição de senha e links de acesso único (magic links) precisam ter prazo de validade curto e precisam funcionar uma única vez — se alguém interceptar um link antigo, ele não pode continuar funcionando indefinidamente. Autenticação em duas etapas (MFA) deveria estar disponível pelo menos para contas administrativas ou de alto privilégio, mesmo que não seja obrigatória para todo mundo. Sair da conta (logout) precisa invalidar a sessão do lado do servidor, não apenas apagar o token guardado no navegador — do contrário, um token roubado antes do logout continua válido mesmo depois. E qualquer rota de login, redefinição de senha, ou qualquer endereço do tipo “tentar de novo” precisa ter limitação de taxa (rate limiting) — sem isso, o sistema fica aberto para tentativas automatizadas de adivinhar senha (força bruta) ou testar listas de senhas vazadas em outros sites (credential stuffing).

Capítulo 9 — Validação de entrada e injeção

Toda validação que realmente importa precisa acontecer do lado do servidor. Validação feita apenas no navegador é uma questão de experiência de uso — ajuda a pessoa a corrigir um erro rápido — mas não é segurança, porque qualquer requisição pode ser enviada diretamente ao servidor sem passar pelo navegador e sem passar por aquela validação visual.

Consultas ao banco de dados sempre precisam usar parametrização ou um mecanismo de mapeamento (ORM) com vinculação de valores — nunca montar a consulta concatenando texto diretamente com valores vindos do usuário. Essa concatenação é a origem clássica de injeção de SQL: se um valor digitado pelo usuário é colado diretamente dentro do comando enviado ao banco, alguém pode digitar, no lugar de um valor comum, um pedaço de comando que altera o significado da consulta inteira — potencialmente lendo ou apagando dados que nunca deveriam ser acessíveis por ali.

Conteúdo exibido em uma página precisa ser escapado por padrão — a maioria dos frameworks modernos já faz isso automaticamente. O risco real está nos mecanismos que desligam essa proteção de propósito (funções do tipo “inserir HTML bruto” disponíveis em praticamente todo framework de interface) usados com conteúdo que vem de um usuário sem

nenhuma sanitização antes. Isso é a porta de entrada para XSS — a injeção de um script malicioso que roda na página de outras pessoas que visualizam aquele conteúdo.

Upload de arquivo merece atenção redobrada: o tipo do arquivo precisa ser validado de verdade no servidor (não apenas pela extensão do nome do arquivo, que é trivialmente falsificável), o tamanho precisa ser limitado, arquivos SVG nunca deveriam ser aceitos sem sanitização (porque um SVG pode conter código de script embutido), e um arquivo enviado por um usuário nunca deveria ser servido de volta no mesmo domínio da aplicação sem isolamento — isso evita que um arquivo malicioso disfarçado de imagem vire um vetor de ataque contra outros usuários.

Qualquer funcionalidade que busca uma URL fornecida por um usuário — um proxy de imagem, uma pré-visualização de link, um endereço de callback configurável — precisa validar que o destino não aponta para um endereço de rede interno ou privado (o clássico ataque de SSRF, onde o próprio servidor é enganado para fazer uma requisição contra sua própria infraestrutura interna, que normalmente não é acessível de fora).

Nunca desserialize um objeto arbitrário vindo do cliente sem um esquema de validação — interpretar um formato de dados estruturado (como JSON) com validação de esquema é seguro; executar código arbitrário vindo do cliente sem nenhum controle não é.

E, para produtos que usam inteligência artificial processando conteúdo de fora (um e-mail, uma página da web, um documento enviado por terceiro): trate esse conteúdo sempre como dado, nunca como instrução. Um texto malicioso escondido dentro desse conteúdo externo — a chamada injeção de prompt — não deveria conseguir fazer o sistema de IA executar uma ação fora do que a pessoa real que está usando o produto pediu.

Capítulo 10 — APIs, webhooks e integrações externas

Qualquer notificação automática recebida de um serviço terceiro (um provedor de pagamento avisando que uma cobrança foi aprovada, um CRM, uma ferramenta de automação) precisa verificar a assinatura ou o segredo daquele provedor antes de confiar em qualquer coisa que veio no corpo da mensagem. Sem essa verificação, qualquer pessoa que descubra o endereço daquele webhook pode forjar um evento falso — o exemplo mais caro na prática é alguém forjar um “pagamento aprovado” falso e liberar acesso pago sem pagar nada.

Se a verificação de um webhook usa um segredo próprio, em vez da assinatura nativa do provedor, esse segredo deveria ser comparado usando uma técnica de comparação em tempo constante quando disponível (para não vazar informação sobre o segredo pela diferença de tempo de resposta), e nunca deveria aparecer em texto puro em nenhum log.

Configuração de CORS — a regra que decide quais domínios podem chamar sua API a partir do navegador — nunca deveria ser “qualquer origem” (*) em um endereço que lê ou escreve dado autenticado; a lista de origens permitidas deveria ser explícita e restrita. Endereços

públicos — webhook, formulário de contato, endpoint de busca — precisam de limitação de taxa, sem a qual eles viram um vetor tanto de negação de serviço quanto de custo direto (imagine um endereço que dispara uma chamada a uma API paga por requisição, sendo bombardeado sem limite).

Endereços de escrita deveriam ser, sempre que possível, idempotentes onde faz sentido — um webhook de pagamento reenviado duas vezes (o que acontece com frequência, por retentativa automática do provedor) não deveria cobrar ou liberar acesso duas vezes. A técnica padrão é usar uma chave de idempotência única, como o identificador do pedido, para reconhecer e ignorar reenvios duplicados.

Toda chamada de saída para uma API de terceiro precisa tratar timeout e erro sem vaziar detalhe técnico interno de volta para o usuário final. E vale um detalhe operacional fácil de esquecer: o domínio do endereço público configurado precisa ser exatamente o mesmo cadastrado no provedor externo — um redirecionamento entre o domínio com e sem “www”, ou entre HTTP e HTTPS, frequentemente não é seguido automaticamente por uma chamada de webhook em POST, o que causa a chamada falhar silenciosamente, sem nenhum log do seu lado indicando o problema.

Capítulo 11 — Dados sensíveis, segredos e criptografia

O arquivo que guarda variáveis de ambiente e credenciais nunca pode ser enviado para o controle de versão — vale confirmar que ele está listado no arquivo de exclusão desde o primeiro commit do repositório. Se esse arquivo já foi enviado alguma vez, mesmo que tenha sido removido depois, a credencial que estava nele já está permanentemente no histórico do repositório — a única correção real é rotacionar (trocar) aquela credencial, apagar o arquivo não resolve nada.

Nenhuma chave privada ou segredo deveria existir numa variável de ambiente que é exposta ao código que roda no navegador (variáveis com um prefixo “público” fazem parte do pacote enviado ao cliente por design, em praticamente qualquer framework moderno) — só chaves genuinamente públicas, como uma chave pública de um SDK feita justamente para ser exposta, deveriam viver ali. Qualquer segredo de um serviço terceiro (token de API, chave privada de integração) nunca deveria ser lido ou repassado diretamente pelo código do navegador — sempre por trás de uma rota do próprio servidor, com a chave guardada numa variável de ambiente disponível só do lado do servidor.

Nenhum dado sensível — senha, número de cartão, documento pessoal — deveria aparecer em log, em mensagem de erro exibida ao usuário, ou em qualquer payload enviado a uma ferramenta de analytics. Dados em trânsito precisam sempre passar por HTTPS/TLS, sem permitir downgrade para uma conexão não criptografada. Dados sensíveis armazenados, quando aplicável, deveriam ter uma camada extra de criptografia no nível da própria coluna ou arquivo — não confiar apenas na criptografia genérica de disco que o provedor de nuvem

oferece por padrão. E qualquer dado pessoal identificável armazenado precisa ter uma política clara de retenção — guardar um dado para sempre “porque sim, nunca vamos precisar apagar” é um risco crescente, não uma economia de esforço.

Capítulo 12 — Vazamento de memória e exaustão de recursos

Este capítulo raramente aparece em checklists de segurança tradicionais, mas é a causa mais comum de um produto “cair sozinho” depois de rodar em produção por alguns dias — e algumas variantes dele viram, na prática, um vetor de negação de serviço explorável por qualquer pessoa.

No lado do servidor, em processos de longa duração: um cache guardado em memória (um objeto que acumula dados enquanto o processo roda) precisa ter um limite de tamanho e/ou uma expiração — um cache que só cresce, indexado por IP ou por usuário, é um vazamento de memória garantido em produção, mesmo que passe despercebido em desenvolvimento. Toda conexão aberta com um banco de dados ou API externa precisa ser sempre fechada e devolvida ao pool de conexões, inclusive no caminho de erro — usar um bloco que garante a liberação mesmo quando algo falha, não apenas o caminho de sucesso. Temporizadores registrados (do tipo “repita a cada X segundos” ou “execute uma vez depois de X segundos”) precisam ser explicitamente cancelados quando o contexto que os criou deixa de existir — um temporizador órfão continua segurando referências na memória indefinidamente. Streams e identificadores de arquivo abertos precisam ser fechados explicitamente depois do uso, mesmo em caso de erro. E qualquer ouvinte de evento registrado dinamicamente (uma conexão em tempo real, um sistema de publicação/assinatura) precisa ser removido quando não é mais necessário — cada ouvinte acumulado segura na memória tudo o que ele referencia no seu contexto interno.

No lado do cliente, em qualquer interface moderna baseada em componentes: todo efeito que registra um ouvinte de evento (um listener na janela, uma assinatura de dados, um temporizador, um observador) precisa ter uma função de limpeza correspondente que desfaz esse registro — sem essa limpeza, cada vez que aquele componente é montado e desmontado na tela, um novo ouvinte se acumula, degradando a página progressivamente com o tempo. Isso é especialmente comum e especialmente danoso em aplicações de sessão longa — painéis administrativos que ficam abertos o dia inteiro, ou uso em dispositivos móveis com pouca memória disponível. Funções internas (closures) não deveriam segurar referência a objetos grandes — arrays de dados extensos, imagens codificadas em texto, respostas completas de API — por mais tempo do que realmente precisam, porque isso impede o coletor de lixo automático da linguagem de liberar aquela memória enquanto a referência existir. Nenhum estado global (um repositório de dados compartilhado, um contexto usado em toda a aplicação) deveria crescer sem limite — um histórico infinito de ações, uma lista de notificações nunca removida, um cache de busca sem expiração. E elementos removidos visualmente da

tela não deveriam deixar uma referência presa em alguma variável esquecida — um elemento “desanexado” da árvore visual, mas ainda referenciado em código, nunca é liberado da memória pelo coletor automático.

Capítulo 13 — Proteção do repositório, cabeçalhos de segurança e infraestrutura

No repositório de código: o arquivo de exclusão do controle de versão precisa cobrir explicitamente o arquivo de variáveis de ambiente e suas variações, com uma exceção explícita apenas para uma versão de exemplo sem valores reais, além de cobrir chaves privadas e pastas de build/dependência. Se o projeto usa contêineres, um arquivo equivalente de exclusão para a construção da imagem também deveria existir espelhando o mesmo conteúdo — sem isso, o arquivo de credenciais e pastas de dependência podem acabar dentro da imagem publicada, mesmo estando corretamente excluídos do controle de versão. Um arquivo de exemplo de variáveis de ambiente, público, sem nenhum valor real — só o nome de cada variável e um texto de exemplo genérico — serve como documentação viva de quais variáveis o sistema precisa, útil tanto para qualquer desenvolvedor novo entrando no projeto quanto para qualquer pipeline automatizado de implantação. E vale ter uma ferramenta de varredura de segredo commitado acidentalmente rodando de forma automática (seja no processo de integração contínua, seja antes de cada commit) — ela pega justamente o caso de alguém enviar acidentalmente um arquivo de credenciais antes de o arquivo de exclusão existir corretamente.

Cabeçalhos de segurança HTTP: uma política de segurança de conteúdo (Content-Security-Policy) deveria estar configurada, mesmo que de forma permissiva no início — o importante é que ela exista e vá sendo refinada com o tempo. O cabeçalho que força conexão sempre criptografada (Strict-Transport-Security, ou HSTS) deveria estar ativo. E os cabeçalhos que impedem que o navegador tente adivinhar o tipo de um arquivo (`X-Content-Type-Options: nosniff`) e que impedem a página de ser carregada dentro de um quadro em outro site (`X-Frame-Options` , ou a diretiva equivalente dentro da própria política de conteúdo) protegem contra ataques de clickjacking — onde um site malicioso disfarça sua página real por trás de uma camada invisível, enganando o clique do usuário. Onde exatamente configurar esses cabeçalhos depende de como a aplicação está hospedada: com um servidor web próprio, eles vão na configuração desse servidor; atrás de uma rede de distribuição de conteúdo ou firewall de aplicação web, podem ser injetados na borda antes mesmo de chegar ao servidor; e em plataformas de hospedagem modernas sem servidor próprio gerenciado diretamente, eles vão na configuração do próprio framework usado para construir a aplicação. Não existe um único lugar “certo” — o que importa é confirmar que os cabeçalhos realmente chegam na resposta enviada ao navegador, de alguma forma.

Comunicação com pesquisadores de segurança de boa-fé: um arquivo padrão chamado `security.txt`, hospedado no caminho bem conhecido `/.well-known/security.txt` (definido por uma especificação técnica aberta, a RFC 9116), informa como qualquer pessoa que encontrar uma vulnerabilidade pode reportá-la de forma responsável, antes de ela ser exposta publicamente. Os campos mínimos são um contato (e-mail ou URL) e uma data de validade do próprio arquivo. O custo de implementação é uma rota estática de poucas linhas; o benefício é dar a alguém de boa-fé um canal claro de comunicação, em vez de forçar essa pessoa a expor a falha publicamente numa rede social por não ter para onde reportar.

Infraestrutura em geral: o arquivo `robots.txt` deveria bloquear a indexação de rotas administrativas ou internas que não deveriam aparecer em resultados de busca — isso não é controle de acesso de verdade (qualquer pessoa pode simplesmente ignorar esse arquivo), mas é uma boa prática básica para não vazarem a existência de um painel interno através de uma busca comum. Dependências do projeto deveriam ter uma varredura automática de vulnerabilidade conhecida rodando continuamente no processo de integração contínua, não apenas verificada manualmente de vez em quando. Um backup do banco de dados precisa existir, precisa ter sido testado de verdade pelo menos uma vez (restaurado num ambiente separado para confirmar que o backup realmente funciona), e não pode ficar guardado no mesmo lugar ou com a mesma credencial do banco principal — do contrário, o mesmo incidente que compromete o banco principal compromete o backup junto. E um firewall de aplicação web ou proteção de borda equivalente na frente dos endereços públicos de maior risco — login, webhook, upload de arquivo — adiciona uma camada extra de defesa antes mesmo da requisição chegar ao código da aplicação.

Capítulo 14 — Erros, logging e observabilidade

Um erro interno — o rastreamento completo da falha, a mensagem bruta vinda do driver do banco de dados, o caminho de um arquivo no servidor — nunca deveria ser devolvido diretamente ao cliente final. A prática correta é sempre devolver uma mensagem genérica e amigável para quem está usando o produto, e registrar o detalhe técnico completo apenas do lado do servidor, onde só o time tem acesso.

Deveria existir um sistema de alerta e monitoramento configurado para capturar erro recorrente automaticamente — sem isso, uma falha silenciosa (por exemplo, uma integração que parou de funcionar) pode passar semanas sem que ninguém perceba, porque nada visível quebrou na experiência do usuário até ser tarde demais. Ações sensíveis — mudança de plano, exclusão de dado, mudança de permissão de alguém — deveriam gerar um registro de auditoria: quem fez, quando, exatamente o quê. E erros de negócio comuns (como “usuário não encontrado” ou “limite atingido”) deveriam ser tratados como distintos de erros reais de sistema (como “banco de dados fora do ar”) dentro dos logs, para que um erro grave de infraestrutura não fique afogado em ruído de eventos esperados do dia a dia.

Um ponto que costuma confundir quem constrói em plataformas modernas de hospedagem sem gerenciar um servidor próprio: não existe um arquivo tradicional de log persistente no sistema de arquivos nesse tipo de ambiente, porque o processo não roda continuamente e o armazenamento local não persiste entre uma execução e outra. O equivalente funcional é: os logs de acesso (quem chamou o quê, quando) vêm da própria plataforma de hospedagem, complementados pelos logs de consulta do banco de dados gerenciado; os logs de autenticação (login, falha de senha, criação de conta, mudança de permissão) precisam de uma tabela de auditoria própria dentro do banco de dados — não vale depender só do que o provedor de autenticação registra por padrão, é importante ter uma tabela consultável a qualquer momento; e os logs de erro vêm de uma ferramenta de monitoramento de exceções configurada, complementada por registros explícitos e estruturados (sempre com contexto: qual rota, qual IP quando aplicável, qual foi o motivo) para eventos de negócio relevantes, como uma tentativa de acesso negada ou um limite de taxa disparado. O importante não é ter exatamente três arquivos físicos separados — é garantir que os três tipos de sinal (acesso, autenticação e erro) sejam pelo menos consultáveis depois que um incidente acontece.

Capítulo 15 — Os erros que passam despercebidos em código gerado rapidamente

Com o uso cada vez mais comum de ferramentas de geração automática de código para acelerar o desenvolvimento, alguns padrões de erro aparecem com frequência desproporcional — não porque a tecnologia seja ruim, mas porque a velocidade de geração cria uma falsa sensação de que “funcionou” é o mesmo que “está seguro”. Análises reais de código gerado dessa forma mostram que vulnerabilidades como injeção de script (XSS) e injeção de dado em logs aparecem numa fração alta do código produzido, e que falhas de autenticação e autorização costumam ser mais frequentes e mais graves nesse tipo de código do que em código escrito manualmente, linha por linha, por alguém prestando atenção deliberada em cada decisão. Isso não é motivo para abandonar ferramentas que aceleram o desenvolvimento — é motivo para sempre auditar deliberadamente o que elas produzem, com atenção redobrada exatamente nestes pontos:

Regras de acesso que “existem mas não protegem” — a regra foi criada porque alguém pediu explicitamente para “ativar a proteção”, mas a condição escrita não restringe nada de verdade (o clássico “sempre verdadeiro”, ou o esquecimento de restringir uma coluna sensível numa operação de edição). Ativar o mecanismo não é o mesmo que configurá-lo corretamente — sempre leia a condição de verdade, nunca assuma pela presença dela.

Validação de posse ausente em funcionalidades de criar/editar/excluir geradas rapidamente em lote — rotas desse tipo, quando geradas em conjunto e depressa, costumam checar apenas “o recurso existe”, sem checar “pertence a quem está pedindo” — o erro de referência direta insegura descrito no Capítulo 7.

Credencial fixa dentro de código de exemplo, dado de teste (seed) ou arquivo de teste automatizado — código gerado dessa forma tem uma taxa alta de credencial fixa embutida diretamente no texto, geralmente justamente em arquivos que parecem inofensivos por serem “só de exemplo” ou “só de teste” — e que depois, sem ninguém perceber, acabam indo parar em produção do mesmo jeito.

Restrição de recurso pago aplicada só na interface — pedir para “esconder essa funcionalidade de quem não pagou” tende a gerar apenas a condição visual no componente de tela, sem replicar a mesma trava no servidor ou na regra do banco de dados, exatamente o problema descrito no Capítulo 7.

Fluxos de reconciliação de estado que só rodam uma vez — um fluxo do tipo “aplique X quando a condição Y acontecer” (por exemplo, aplicar um crédito pendente no momento da criação de uma conta nova) costuma ser implementado apenas no caminho “feliz” mais óbvio (a criação da conta), esquecendo o caminho onde a conta já existia antes — deixando casos legítimos permanentemente travados, sem gerar nenhum erro visível que chame atenção para o problema.

Webhook sem verificação de assinatura — é comum pedir “crie um endereço que recebe a notificação automática de tal serviço” e receber de volta um manipulador que confia cegamente em qualquer conteúdo recebido, sem checar segredo ou assinatura nenhuma. Isso costuma passar despercebido porque os próprios testes automatizados, quando existem, também não simulam o envio de uma assinatura real — e o problema só aparece exposto em produção.

Limitação de taxa esquecida por padrão — raramente uma ferramenta de geração automática adiciona limitação de taxa sem ser explicitamente instruída a fazer isso, deixando endereços de login, webhook e formulário público abertos a abuso sistemático.

Condição de corrida no padrão “verifica e depois age” — um padrão comum e perigoso é “consultar o limite atual, e se estiver dentro do limite, inserir o novo registro” em duas etapas separadas. Sob duas requisições simultâneas, ambas podem passar pela etapa de verificação antes que qualquer uma delas complete a etapa de inserção — furando um limite que deveria ser absoluto (por exemplo, permitindo dois usos de um cupom de uso único). A prevenção correta é sempre no nível do banco de dados — uma restrição de unicidade, uma restrição de validação, ou uma transação atômica — nunca confiar apenas numa lógica de duas etapas separadas dentro do código da aplicação.

Dependência sugerida que não existe de verdade ou tem nome parecido com um pacote malicioso real — ferramentas de geração automática às vezes sugerem instalar um pacote de código de terceiro que não existe, ou que tem um nome muito parecido com um pacote oficial legítimo (uma técnica de ataque conhecida como confusão de dependências, ou typosquatting). Sempre confira que o pacote sugerido realmente existe, é o pacote oficial esperado, e tem uso e manutenção ativa reais antes de instalar qualquer coisa.

Liberar CORS totalmente “para resolver rápido” — quando um erro de política de origem cruzada trava o desenvolvimento local, a correção mais rápida e mais tentadora costuma ser liberar qualquer origem (*) — fácil de esquecer de restringir depois, antes de o produto ir para produção de verdade.

E, por fim, assumir que “passou no teste” é o mesmo que “está seguro” — testes automatizados comuns normalmente não simulam um atacante tentando ativamente furar o sistema (tentando outro identificador, outro token, um dado malformado de propósito). Passar em todos os testes normais não é evidência de estar protegido contra alguém agindo deliberadamente para quebrar o sistema — são duas perguntas completamente diferentes.

Capítulo 16 — Checklist rápido de pré-produção

Antes de considerar qualquer produto pronto para receber usuários reais, dinheiro real ou dado sensível real, confirme cada item abaixo:

- ☐ O controle de acesso foi testado tentando de propósito acessar o dado de outro usuário — não só confirmando que o próprio usuário consegue acessar o próprio dado.
 - ☐ Nenhum segredo aparece em arquivo de ambiente commitado, em variável exposta ao cliente, ou em qualquer log.
 - ☐ Existe limitação de taxa em login, em webhook e em qualquer formulário público.
 - ☐ Todo webhook recebido de terceiro valida assinatura ou segredo antes de confiar no conteúdo.
 - ☐ Nenhum erro interno chega até o cliente final em texto bruto.
 - ☐ Os cabeçalhos de segurança básicos (política de conteúdo, HSTS) estão configurados.
 - ☐ As dependências do projeto têm varredura automática de vulnerabilidade rodando continuamente.
 - ☐ Existe backup do banco de dados, e ele já foi restaurado pelo menos uma vez só para testar que funciona de verdade.
 - ☐ A restrição de plano ou permissão é aplicada no servidor, nunca apenas escondida na interface visual.
 - ☐ Nenhum cache ou estado global cresce sem limite, nem no servidor nem no cliente.
 - ☐ Todo ouvinte de evento, temporizador ou assinatura tem uma limpeza correspondente registrada.
 - ☐ Existe monitoramento de erro ativo, com alerta configurado, não só passivo.
-

CHECKLIST CONSOLIDADO — ANTES DE LANÇAR

Este é o checklist final, combinando as duas partes. Ele não substitui os capítulos anteriores — cada item aqui é um resumo de algo já explicado em profundidade. Use este checklist como a última passagem antes de anunciar o lançamento, e trate qualquer item não marcado como um bloqueador, não como um “depois a gente ajusta”.

Design e conversão

- ☐ O design tem ponto de vista próprio — não é copiável e colável por um concorrente sem perder sentido.
- ☐ Paleta com no máximo três a cinco cores, hierarquia tipográfica clara entre título, subtítulo e corpo.
- ☐ Espaço em branco generoso; nenhuma seção está “espremida” por pressa.
- ☐ Nenhuma imagem de banco de imagens genérica; onde não há imagem real, não há imagem nenhuma.
- ☐ Toda animação tem um propósito de comunicação, não é decoração.
- ☐ O layout mobile foi desenhado como versão própria e testado num dispositivo real.
- ☐ Contraste de texto pelo menos AA, HTML semântico, performance real, meta tags e imagem de compartilhamento configuradas.
- ☐ A seção principal (hero) vende sozinha, com uma única chamada para ação por tela.
- ☐ O texto do botão descreve o próximo passo, não uma palavra vaga.
- ☐ Existe pelo menos uma prova social real antes do lançamento.
- ☐ A copy usa números concretos, elimina palavras fracas, e não poderia ser colada por um concorrente sem soar estranha.
- ☐ O produto consegue ser descrito em menos de dez palavras.

Segurança

- ☐ Toda tabela com dado de usuário tem regra de acesso restritiva testada de verdade — nunca uma condição sempre-verdadeira.
- ☐ Colunas sensíveis (plano, saldo, permissão) não podem ser editadas livremente pelo próprio usuário.
- ☐ Nenhuma referência direta insegura a objeto — trocar um ID na URL nunca revela dado de outra pessoa.

- ☐ Nenhuma restrição de acesso existe só na interface visual — toda trava real está no servidor.
- ☐ Token de sessão em cookie seguro, nunca em armazenamento livremente legível por script.
- ☐ Senhas com hash correto (bcrypt/argon2), nunca texto puro ou criptografia reversível.
- ☐ Limitação de taxa em login, redefinição de senha, webhook e formulário público.
- ☐ Toda consulta ao banco usa parametrização — nunca concatenação de texto.
- ☐ Upload de arquivo validado por tipo real, tamanho limitado, SVG sanitizado.
- ☐ Todo webhook de terceiro valida assinatura antes de confiar no payload.
- ☐ Nenhum segredo em arquivo de ambiente commitado, em variável exposta ao cliente, ou em log.
- ☐ Erro interno nunca chega ao cliente final em texto bruto; monitoramento de erro ativo com alerta.
- ☐ Cabeçalhos de segurança configurados; dependências com varredura automática de vulnerabilidade.
- ☐ Backup do banco existe e já foi restaurado pelo menos uma vez para testar.
- ☐ Nenhum cache, estado global, listener ou temporizador cresce ou vaza sem limite.
- ☐ O controle de acesso foi testado ativamente tentando acessar dado de outra conta — não só confirmando o caminho feliz.

Um produto pronto para lançar é aquele em que cada linha acima pode ser respondida com uma evidência concreta, não com uma impressão. “Acho que está bom” não é uma resposta válida para nenhum item desta lista — cada um foi escrito para ser verificado, não assumido.